



Vidyavardhini's College of Engineering and Technology

Department of Artificial Intelligence & Data Science

AY: 2026-27

Class:	TE-AI&DS	Semester:	V
Course Code:	2015111	Course Name:	Statistics for Machine Learning and Data Science Lab

Name of Student:	Bhavik Vijay Gharat
Roll No. :	69
Experiment No.:	7
Title of the Experiment:	Build and Evaluate Classification Models using Supervised Learning Algorithms
Date of Performance:	
Date of Submission:	

Evaluation

Performance Indicator	Max. Marks	Marks Obtained
Performance	5	
Understanding	5	
Journal work and timely submission	10	
Total	20	

Performance Indicator	Exceed Expectations (EE)	Meet Expectations (ME)	Below Expectations (BE)
Performance	4-5	2-3	1
Understanding	4-5	2-3	1
Journal work and timely submission	8-10	5-8	1-4

Checked by

Name of Faculty :

Signature :

Date :



Vidyavardhini's College of Engineering and Technology

Department of Artificial Intelligence & Data Science

Aim: To build supervised classification models for predicting diabetes outcomes using the Pima Indians Diabetes Dataset and evaluate their performance using appropriate classification metrics.

Objectives

After completing this experiment, the student will be able to:

1. Build supervised classification models to predict the diabetes outcome of patients.
2. Evaluate and interpret classification model performance using appropriate evaluation metrics.

Tools Required

- Python 3.x
- Google Colab / Jupyter Notebook
- Pandas
- NumPy
- Matplotlib
- Seaborn
- Scikit-learn

Dataset Details

Dataset: Pima Indians Diabetes Dataset

The dataset contains medical information for **768 female patients**.

The target variable is:

- Outcome = 0: Non-diabetic
- Outcome = 1: Diabetic



Vidyavardhini's College of Engineering and Technology

Department of Artificial Intelligence & Data Science

Procedure

1. Load the Pima Indians Diabetes Dataset.
2. Separate the input features and target variable.
3. Identify and handle invalid or missing values.
4. Divide the dataset into training and testing sets using a stratified split.
5. Apply suitable preprocessing, such as feature scaling where required.
6. Build one or more classification models, such as:
 - Logistic Regression
 - k-Nearest Neighbors
 - Decision Tree
7. Train the selected model(s) using the training data.
8. Generate predictions for the test data.
9. Construct the confusion matrix.
10. Calculate classification performance metrics.
11. Compare the obtained results and interpret the model performance.

Description of the Experiment

Classification is a supervised learning technique used to assign observations to predefined categories.

In this experiment, the objective is to classify a patient as: Patient Health Data, Classification Model gives Non-Diabetic and Diabetic.

The experiment follows: Dataset, Feature and Target Selection, Data Preprocessing, Train-Test Split, Classification Model, Prediction, and Performance Evaluation



Detailed Description of Statistical Methods

Logistic Regression

Logistic regression predicts the probability of a binary outcome.

The logistic function is:

$$P(Y = 1) = \frac{1}{1 + e^{-z}}$$

where:

$$z = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots + \beta_p x_p$$

The predicted probability is converted into a class using a decision threshold.

Confusion Matrix

A confusion matrix contains:

Predicted Negative Predicted Positive

Actual Negative True Negative (TN) False Positive (FP)

Actual Positive False Negative (FN) True Positive (TP)

Accuracy

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

Precision

$$Precision = \frac{TP}{TP + FP}$$

Precision indicates how many predicted positive cases were actually positive.

Recall

$$Recall = \frac{TP}{TP + FN}$$

Recall indicates how many actual positive cases were correctly identified.



F1-Score

$$F1 = 2 \left(\frac{Precision \times Recall}{Precision + Recall} \right)$$

The F1-score balances precision and recall.

ROC-AUC

The ROC curve represents the relationship between:

- True Positive Rate
- False Positive Rate

The Area Under the Curve (AUC) provides an overall measure of the model's ability to distinguish between the two classes.

Code:-

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler

from sklearn.linear_model import LogisticRegression
from sklearn.neighbors import KNeighborsClassifier
from sklearn.tree import DecisionTreeClassifier

from sklearn.metrics import (
    confusion_matrix,
```



Vidyavardhini's College of Engineering and Technology

Department of Artificial Intelligence & Data Science

```
accuracy_score,  
precision_score,  
recall_score,  
f1_score,  
roc_auc_score,  
roc_curve,  
classification_report  
)  
  
df = pd.read_csv("diabetes.csv")  
  
  
print("=" * 60)  
print("PIMA INDIANS DIABETES DATASET")  
print("=" * 60)  
  
  
print("\nFirst 5 rows:")  
print(df.head())  
  
  
print("\nDataset Shape:")  
print(df.shape)  
  
  
print("\nColumn Names:")  
print(df.columns)  
  
  
print("\nDataset Information:")  
print(df.info())  
  
  
print("\nStatistical Summary:")
```



```
print(df.describe())

print("\n" + "=" * 60)

print("MISSING VALUES")

print("=" * 60)


print(df.isnull().sum())

X = df.drop("Outcome", axis=1)

y = df["Outcome"]


print("\nInput Features:")

print(X.columns.tolist())


print("\nTarget Variable:")

print("Outcome")

invalid_columns = [

    "Glucose",

    "BloodPressure",

    "SkinThickness",

    "Insulin",

    "BMI"

]

X[invalid_columns] = X[invalid_columns].replace(0, np.nan)


print("\nInvalid values replaced with NaN.")


print("\nMissing values after replacing invalid zeros:")

2015111: Statistics for Machine Learning and Data Science Lab
```



Vidyavardhini's College of Engineering and Technology

Department of Artificial Intelligence & Data Science

```
print(X.isnull().sum())

imputer = SimpleImputer(strategy="median")

X = pd.DataFrame(
    imputer.fit_transform(X),
    columns=X.columns
)

print("\nMissing values after median imputation:")
print(X.isnull().sum())

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42,
    stratify=y
)

print("\n" + "=" * 60)
print("TRAIN-TEST SPLIT")
print("=" * 60)

print("Training samples:", X_train.shape[0])
print("Testing samples :", X_test.shape[0])

scaler = StandardScaler()
```



Vidyavardhini's College of Engineering and Technology

Department of Artificial Intelligence & Data Science

```
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

print("\nFeature scaling completed.")

logistic_model = LogisticRegression(max_iter=1000)

knn_model = KNeighborsClassifier(n_neighbors=5)

decision_tree_model = DecisionTreeClassifier(
    random_state=42,
    max_depth=5
)

knn_model.fit(X_train_scaled, y_train)
decision_tree_model.fit(X_train, y_train)

print("\nAll models trained successfully.")

y_pred_lr = logistic_model.predict(X_test_scaled)
y_prob_lr = logistic_model.predict_proba(X_test_scaled)[: , 1]
y_pred_knn = knn_model.predict(X_test_scaled)
y_prob_knn = knn_model.predict_proba(X_test_scaled)[: , 1]
y_pred_dt = decision_tree_model.predict(X_test)
y_prob_dt = decision_tree_model.predict_proba(X_test)[: , 1]

def evaluate_model(model_name, y_test, y_pred, y_prob):

    print("\n")
```



Vidyavardhini's College of Engineering and Technology

Department of Artificial Intelligence & Data Science

```
print("=" * 60)
print(model_name)
print("=" * 60)
cm = confusion_matrix(y_test, y_pred)

print("\nConfusion Matrix:")
print(cm)
TN, FP, FN, TP = cm.ravel()

print("\nTN =", TN)
print("FP =", FP)
print("FN =", FN)
print("TP =", TP)

accuracy = accuracy_score(y_test, y_pred)
precision = precision_score(
    y_test,
    y_pred,
    zero_division=0
)
recall = recall_score(
    y_test,
    y_pred,
    zero_division=0
)
f1 = f1_score(
    y_test,
```



```
y_pred,  
zero_division=0  
)  
  
auc = roc_auc_score(y_test, y_prob)  
  
print("\nPerformance Metrics:")  
print("Accuracy :", round(accuracy, 4))  
print("Precision :", round(precision, 4))  
print("Recall   :", round(recall, 4))  
print("F1-Score  :", round(f1, 4))  
print("ROC-AUC   :", round(auc, 4))  
  
print("\nClassification Report:")  
print(  
    classification_report(  
        y_test,  
        y_pred,  
        target_names=["Non-Diabetic", "Diabetic"],  
        zero_division=0  
    )  
)  
  
return accuracy, precision, recall, f1, auc  
  
lr_results = evaluate_model(  
    "LOGISTIC REGRESSION",
```



Vidyavardhini's College of Engineering and Technology

Department of Artificial Intelligence & Data Science

```
y_test,  
y_pred_lr,  
y_prob_lr  
)  
  
knn_results = evaluate_model(  
    "K-NEAREST NEIGHBORS (KNN)",  
    y_test,  
    y_pred_knn,  
    y_prob_knn  
)  
  
dt_results = evaluate_model(  
    "DECISION TREE",  
    y_test,  
    y_pred_dt,  
    y_prob_dt  
)  
  
results = pd.DataFrame({  
  
    "Model": [  
        "Logistic Regression",  
        "KNN",  
        "Decision Tree"  
    ],  
  
    "Accuracy": [  

```



Vidyavardhini's College of Engineering and Technology

Department of Artificial Intelligence & Data Science

```
lr_results[0],  
knn_results[0],  
dt_results[0]  
],
```

```
"Precision": [  
    lr_results[1],  
    knn_results[1],  
    dt_results[1]  
],
```

```
"Recall": [  
    lr_results[2],  
    knn_results[2],  
    dt_results[2]  
],
```

```
"F1-Score": [  
    lr_results[3],  
    knn_results[3],  
    dt_results[3]  
],
```

```
"ROC-AUC": [  
    lr_results[4],  
    knn_results[4],  
    dt_results[4]  
]
```



Vidyavardhini's College of Engineering and Technology

Department of Artificial Intelligence & Data Science

```
]
})

print("\n")
print("=" * 80)
print("MODEL COMPARISON")
print("=" * 80)

print(results.round(4).to_string(index=False))

fpr_lr, tpr_lr, _ = roc_curve(y_test, y_prob_lr)
fpr_knn, tpr_knn, _ = roc_curve(y_test, y_prob_knn)
fpr_dt, tpr_dt, _ = roc_curve(y_test, y_prob_dt)

plt.figure(figsize=(8, 6))

plt.plot(
    fpr_lr,
    tpr_lr,
    label="Logistic Regression"
)

plt.plot(
    fpr_knn,
    tpr_knn,
    label="KNN"
```



Vidyavardhini's College of Engineering and Technology

Department of Artificial Intelligence & Data Science

)

```
plt.plot(  
    fpr_dt,  
    tpr_dt,  
    label="Decision Tree"
```

```
)
```

```
plt.plot(  
    [0, 1],  
    [0, 1],  
    linestyle="--",  
    label="Random Classifier"
```

```
)
```

```
plt.xlabel("False Positive Rate")  
plt.ylabel("True Positive Rate")
```

```
plt.title("ROC Curve - Diabetes Classification")
```

```
plt.legend()
```

```
plt.grid()
```

```
plt.show()
```



Vidyavardhini's College of Engineering and Technology

Department of Artificial Intelligence & Data Science

```
models_cm = {  
    "Logistic Regression": confusion_matrix(y_test, y_pred_lr),  
    "KNN": confusion_matrix(y_test, y_pred_knn),  
    "Decision Tree": confusion_matrix(y_test, y_pred_dt)  
}
```

```
for name, cm in models_cm.items():
```

```
    plt.figure(figsize=(5, 4))
```

```
    plt.imshow(cm)
```

```
    plt.title("Confusion Matrix - " + name)
```

```
    plt.xlabel("Predicted Label")
```

```
    plt.ylabel("Actual Label")
```

```
    plt.xticks(  
        [0, 1],  
        ["Non-Diabetic", "Diabetic"]  
    )
```

```
    plt.yticks(  
        [0, 1],  
        ["Non-Diabetic", "Diabetic"]  
    )
```



Vidyavardhini's College of Engineering and Technology

Department of Artificial Intelligence & Data Science

```
for i in range(2):
```

```
    for j in range(2):
```

```
        plt.text(
```

```
            j,
```

```
            i,
```

```
            cm[i, j],
```

```
            ha="center",
```

```
            va="center"
```

```
        )
```

```
plt.colorbar()
```

```
plt.show()
```

```
print("\n")
```

```
print("=" * 80)
```

```
print("FINAL RESULT")
```

```
print("=" * 80)
```

```
print("""
```

The Pima Indians Diabetes Dataset was successfully classified
using Logistic Regression, KNN and Decision Tree.

The models were evaluated using:

1. Confusion Matrix
2. Accuracy
3. Precision



4. Recall
5. F1-Score
6. ROC-AUC

The model comparison table shows the performance of all classification algorithms.

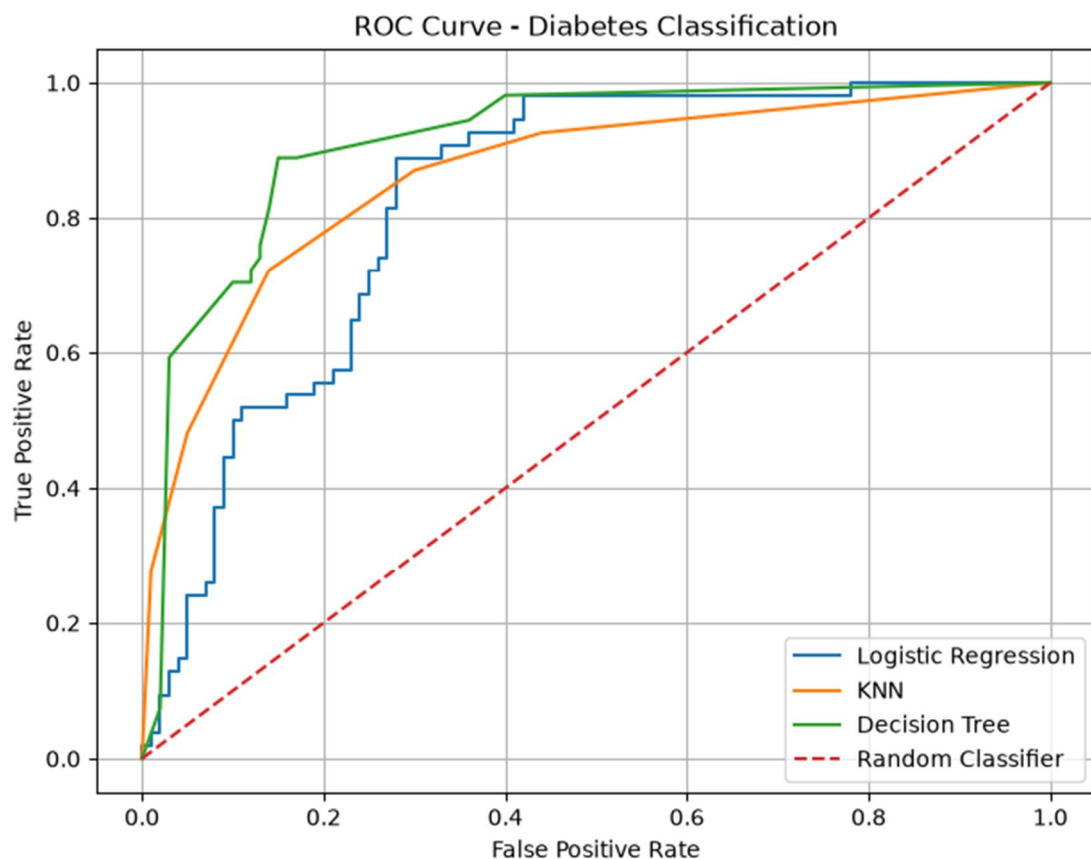
""")

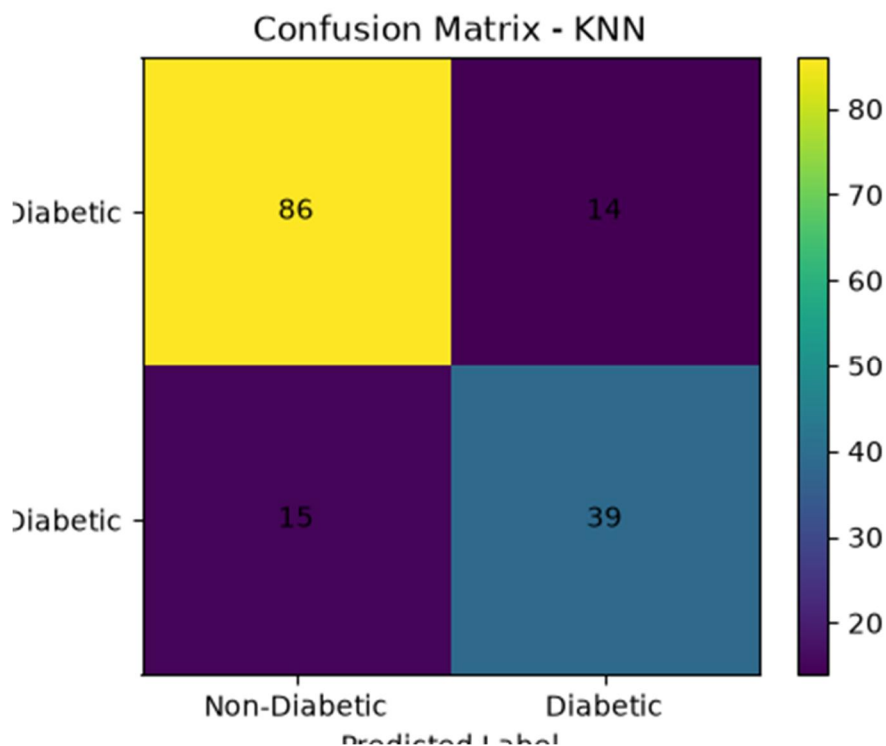
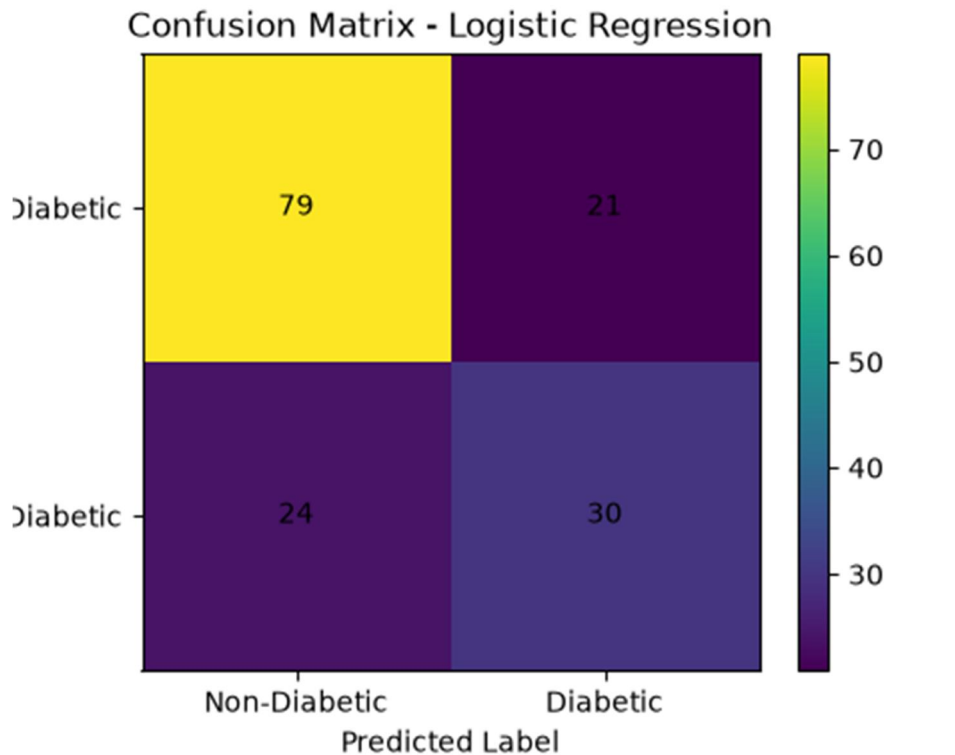
```
print("=" * 80)
```

```
print("EXPERIMENT COMPLETED SUCCESSFULLY")
```

```
print("=" * 80)
```

Output:-

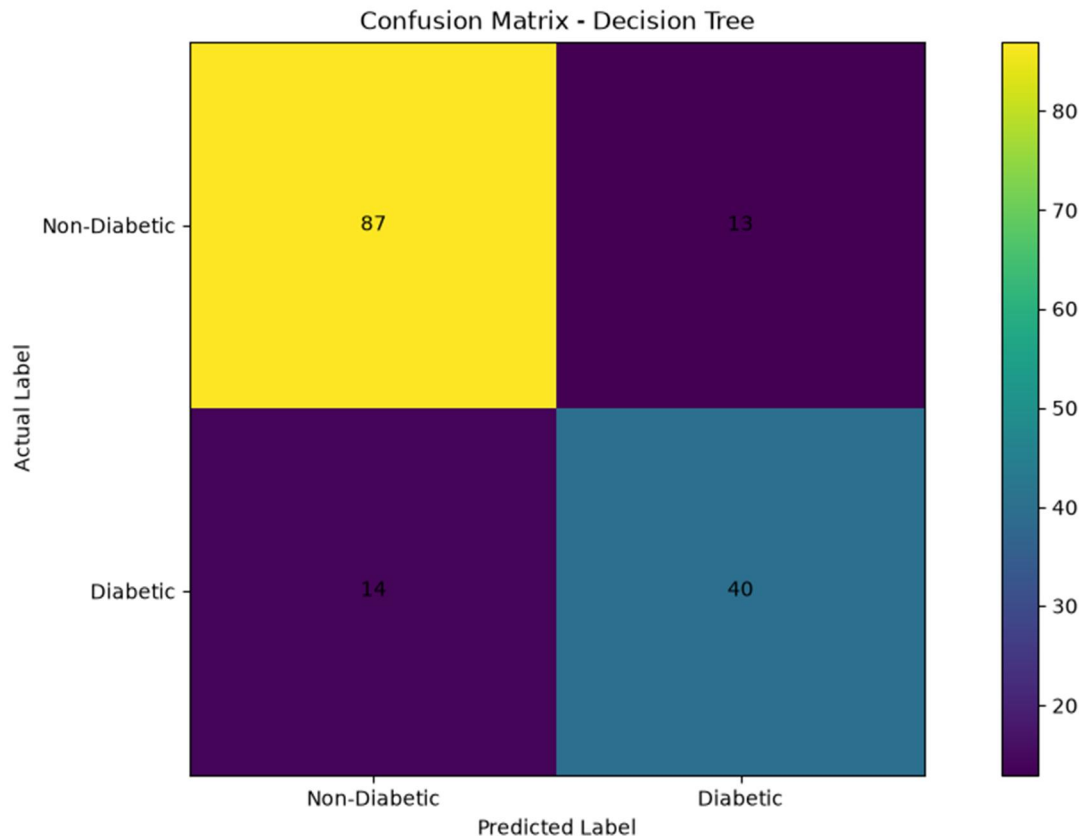






Vidyavardhini's College of Engineering and Technology

Department of Artificial Intelligence & Data Science



=====

PIMA INDIANS DIABETES DATASET

=====

First 5 rows:

	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI	DiabetesPedigreeFunction	Age	Outcome
0	6	148	72.0	35	169.5	33.6	0.627	50	1
1	1	85	66.0	29	102.5	26.6	0.351	31	0
2	8	183	64.0	32	169.5	23.3	0.672	32	1
3	1	89	66.0	23	94.0	28.1	0.167	21	0
4	0	137	40.0	35	168.0	43.1	2.288	33	1

Dataset Shape:
(768, 9)

Column Names:

```
Index(['Pregnancies', 'Glucose', 'BloodPressure', 'SkinThickness', 'Insulin',  
       'BMI', 'DiabetesPedigreeFunction', 'Age', 'Outcome'],  
      dtype='str')
```

Dataset Information:

<class 'pandas.DataFrame'>

RangeIndex: 768 entries, 0 to 767

Data columns (total 9 columns):

#	Column	Non-Null Count	Dtype
0	Pregnancies	768 non-null	int64
1	Glucose	768 non-null	int64
2	BloodPressure	768 non-null	float64
3	SkinThickness	768 non-null	int64
4	Insulin	768 non-null	float64
5	BMI	768 non-null	float64
6	DiabetesPedigreeFunction	768 non-null	float64
7	Age	768 non-null	int64
8	Outcome	768 non-null	int64

dtypes: float64(4), int64(5)

memory usage: 54.1 KB

None



Vidyavardhini's College of Engineering and Technology

Department of Artificial Intelligence & Data Science

```
Statistical Summary:
Pregnancies      Glucose      BloodPressure      ...      DiabetesPedigreeFunction      Age      Outcome
count      768.000000      768.000000      768.000000      ...      768.000000      768.000000      768.000000
mean      3.845052      121.677083      72.389323      ...      0.471876      33.240885      0.348958
std      3.369578      30.464161      12.106039      ...      0.331329      11.760232      0.476951
min      0.000000      44.000000      24.000000      ...      0.078000      21.000000      0.000000
25%      1.000000      99.750000      64.000000      ...      0.243750      24.000000      0.000000
50%      3.000000      117.000000      72.000000      ...      0.372500      29.000000      0.000000
75%      6.000000      140.250000      80.000000      ...      0.626250      41.000000      1.000000
max      17.000000      199.000000      122.000000      ...      2.420000      81.000000      1.000000

[8 rows x 9 columns]

=====
MISSING VALUES
=====
Pregnancies      0
Glucose      0
BloodPressure      0
SkinThickness      0
Insulin      0
BMI      0
DiabetesPedigreeFunction      0
Age      0
Outcome      0
dtype: int64

Input Features:
['Pregnancies', 'Glucose', 'BloodPressure', 'SkinThickness', 'Insulin', 'BMI', 'DiabetesPedigreeFunction', 'Age']

Target Variable:
Outcome

Invalid values replaced with NaN.

Missing values after replacing invalid zeros:
Pregnancies      0
Glucose      0
BloodPressure      0
SkinThickness      0
Insulin      0
BMI      0
DiabetesPedigreeFunction      0
Age      0
dtype: int64

Missing values after median imputation:
Pregnancies      0
Glucose      0
BloodPressure      0
SkinThickness      0
Insulin      0
BMI      0
DiabetesPedigreeFunction      0
Age      0
dtype: int64

=====
TRAIN-TEST SPLIT
=====
Training samples: 614
Testing samples : 154

Feature scaling completed.

All models trained successfully.
```



LOGISTIC REGRESSION

Confusion Matrix:

```
[[79 21]
 [24 30]]
```

TN = 79

FP = 21

FN = 24

TP = 30

Performance Metrics:

Accuracy : 0.7078

Precision : 0.5882

Recall : 0.5556

F1-Score : 0.5714

ROC-AUC : 0.8263

Classification Report:

	precision	recall	f1-score	support
Non-Diabetic	0.77	0.79	0.78	100
Diabetic	0.59	0.56	0.57	54
accuracy			0.71	154
macro avg	0.68	0.67	0.67	154
weighted avg	0.70	0.71	0.71	154

K-NEAREST NEIGHBORS (KNN)

Confusion Matrix:

```
[[86 14]
 [15 39]]
```

TN = 86

FP = 14

FN = 15

TP = 39

Performance Metrics:

Accuracy : 0.8117

Precision : 0.7358

Recall : 0.7222

F1-Score : 0.729

ROC-AUC : 0.8631

Classification Report:

	precision	recall	f1-score	support
Non-Diabetic	0.85	0.86	0.86	100
Diabetic	0.74	0.72	0.73	54
accuracy			0.81	154
macro avg	0.79	0.79	0.79	154
weighted avg	0.81	0.81	0.81	154



DECISION TREE

Confusion Matrix:

```
[[87 13]
 [14 40]]
```

TN = 87

FP = 13

FN = 14

TP = 40

Performance Metrics:

Accuracy : 0.8247

Precision : 0.7547

Recall : 0.7407

F1-Score : 0.7477

ROC-AUC : 0.9121

Classification Report:

	precision	recall	f1-score	support
Non-Diabetic	0.86	0.87	0.87	100
Diabetic	0.75	0.74	0.75	54
accuracy			0.82	154
macro avg	0.81	0.81	0.81	154
weighted avg	0.82	0.82	0.82	154

MODEL COMPARISON

Model	Accuracy	Precision	Recall	F1-Score	ROC-AUC
Logistic Regression	0.7078	0.5882	0.5556	0.5714	0.8263
KNN	0.8117	0.7358	0.7222	0.7290	0.8631
Decision Tree	0.8247	0.7547	0.7407	0.7477	0.9121

FINAL RESULT

The Pima Indians Diabetes Dataset was successfully classified using Logistic Regression, KNN and Decision Tree.

The models were evaluated using:

1. Confusion Matrix
2. Accuracy
3. Precision
4. Recall
5. F1-Score
6. ROC-AUC

The model comparison table shows the performance of all classification algorithms.

EXPERIMENT COMPLETED SUCCESSFULLY



Conclusion

Supervised classification models were developed to predict diabetes outcomes using the Pima Indians Diabetes Dataset. The models were evaluated using the confusion matrix, accuracy, precision, recall, F1-score, and ROC-AUC. The experiment demonstrated that different evaluation metrics provide different perspectives on classification performance.

Questions to be Answered by Students

1. Which classification model was implemented, and what were its accuracy, precision, recall, and F1-score?

Three classification models were implemented:

Model	Accuracy	Precision	Recall	F1-Score
Logistic Regression	0.7078	0.5882	0.5556	0.5714
KNN	0.8117	0.7358	0.7222	0.7290
Decision Tree	0.8247	0.7547	0.7407	0.7477

For the **KNN model**, the values are:

- Accuracy = **0.8117 (81.17%)**
- Precision = **0.7358 (73.58%)**
- Recall = **0.7222 (72.22%)**
- F1-Score = **0.7290 (72.90%)**

2. How many true positives and false negatives were obtained from the confusion matrix?

For the KNN model, the confusion matrix is:

[[86 14]

[15 39]]

Therefore:



- True Positive (TP) = 39
- False Negative (FN) = 15

So, 39 diabetic patients were correctly identified, while 15 diabetic patients were incorrectly classified as non-diabetic.

3. Which metric is more important for diabetes prediction: precision or recall? Justify your answer.

Recall is generally more important for diabetes prediction.

The KNN model has a recall of 0.7222 (72.22%).

Recall is important because it measures how many actual diabetic patients are correctly identified. A false negative means a diabetic patient is classified as non-diabetic, which could result in the patient not receiving further medical evaluation.

Therefore, a higher recall helps reduce missed diabetic cases.

4. What was the ROC-AUC value, and what does it indicate?

For the KNN model:

ROC-AUC = 0.8631

An ROC-AUC of **0.8631** indicates that the model has a good ability to distinguish between diabetic and non-diabetic patients.

The ROC-AUC values from your models are:

- Logistic Regression = **0.8263**
- KNN = **0.8631**
- Decision Tree = **0.9121**

5. State two limitations of the developed classification model.

1. **Limited dataset:** The model is trained using only 768 patients, so its results may not generalize to every population.
2. **Limited features:** The model uses only the medical features available in the dataset. Other factors such as diet, physical activity, detailed family history, and additional medical tests are not included.